

# Final Project Report for CS 175

**Project Title:** DiabetesRL

**Project Number:** 10

**Student Name:** Neda Mohseni, Nmohsen1, nmohsen1@uci.edu

## 1. Introduction and Problem Statement

This project addresses the problem of automated insulin dosing for individuals with Type 1 diabetes using reinforcement learning. Diabetes patients need insulin injection to keep blood glucose in safe range. The task is challenging because glucose–insulin regulation is nonlinear, delayed, and only partially observable: at each timestep, the agent receives a noisy observation derived from the CGM (continuous glucose monitoring) sensor, while **the patient's physiological parameters and true blood glucose level remain hidden from the agent**. The experiments are conducted in SimGlucose (Xie, 2018)[1], a simulated environment for Type 1 diabetes control that provides virtual patients, meal events, and insulin actions. The objective is to learn a policy that selects insulin infusion rates to keep the patient in a safe blood glucose range (70-180 mg/dl) for as long as possible, reduce hypoglycemia (low blood glucose < 70 mg/dl) and hyperglycemia (high blood glucose > 180 mg/dl).

To study this problem, the project trains and evaluates three PPO-based agents: (1) a vanilla PPO agent that uses only the current observation vector, (2) a history-augmented PPO that receives the past few observation vectors. Agents are trained on a single adult virtual patient and evaluated by survival time and stability. Additional tests on adolescent and child patients assess cross-patient generalization. The main result is experimenting whether incorporating temporal context improves policy stability and survival in this partially observable control setting.

## 2. Related Work:

Reinforcement learning has been used extensively for automated insulin dosing in simulators like **SimGlucose**, with methods such as PPO, A2C, SAC, DQN, and DDQN, and many more advanced models applied to this control problem. These approaches aim to manage partial observability, delayed insulin effect, and variability across virtual patients. However, due to the delayed effect of insulin and partial observability, it's critical for the agent to know past trends. Our project builds on this line of work by evaluating PPO-based agents and studying how temporal context affects performance. Below is a summary of the two “most relevant” papers I found.

### **Paper 1. Explainable Recurrent PPO : Determining Proper Insulin Dosage (Youssefpour et al., 2025) [2]**

This paper evaluates a PPO baseline and then extends it with recurrent PPO by incorporating an LSTM module to capture temporal dependencies. The LSTM enables the agent to retain information about past observations and better account for the delayed effect of insulin, improving decision-making in a partially observable environment. The authors further augment the recurrent PPO model with a feature-level attention mechanism, added on top of the LSTM architecture, to highlight which input features were most influential at each timestep. meal info was found to be the most influential feature. This is the only paper found using recurrent PPO for this task, though it is not a peer-reviewed study.

## Paper 2. G2P2C - A modular rl algorithm for glucose control by glucose prediction and planning in Type 1 Diabetes. (Hettiarachchi et al., 2023)[3]

This paper extends PPO to improve performance and safety by focusing on short-term outcomes. By introducing two optimization phases: (1) a model-learning phase, where a glucose dynamic model is learned as an auxiliary task, and (2) a planning phase, where the learned control policy is fine-tuned over a short time horizon using model-based simulations to improve safety. Their observation space includes the CGM value along with insulin delivered in the past  $k$  steps (with  $k = 12$ , representing one hour at a 5-minute sampling rate). The authors believe fully automated glucose control should not require meal announcements or carbohydrate estimation, therefore, they do not include meal info in the observations

### 3. Data Sets

**Environment :** [SimGlucose](#) is an FDA-approved Type 1 Diabetes simulation environment designed for reinforcement learning, following the OpenAI Gym/Gymnasium and rllab APIs. It is built on top of the UVA/Padova Type 1 Diabetes Simulator, a clinically validated physiological model widely used in diabetes research and algorithm testing.

It provides 30 virtual patients : 10 adults, 10 adolescents, and 10 children. Each patient has unique physiological parameters, reflecting differences in insulin sensitivity, glucose dynamics, and metabolic behavior. These parameters define the internal latent physiology of each virtual patient and are hidden from the agent. [Here](#) is a full list of these parameters. All of our training is done on a single adult virtual patient (Adult#001).

The environment simulates how virtual patients respond over time to meals (carbohydrate intake), insulin dosing and patient physiological variation. Each episode is a 24-hour simulation, with each step representing 3 minutes, for a total of 480 timesteps.

The environment is highly stochastic due to meal disturbances, the blood-glucose sensor model, and the insulin pump, both of which can be customized or randomized. To reduce stochasticity, we use a fixed daily meal scenario starting at 6:00 AM, with meals at 7:00 AM (45 g, breakfast), 12:00 PM (70 g, lunch), 4:00 PM (15 g), 6:00 PM (80 g, dinner), and 11:00 PM (10 g), with the meal amounts inspired by the example setup in the SimGlucose README

**Observations :** Default environment: SimGlucose provides a single continuous CGM glucose value (mg/dL) at each timestep, typically ranging from 10 to 500. We divide the CGM value by 400 for normalization so that it falls roughly within  $[0,1]$ .

**Feature wrapper:** to handle partial observability, we augment the observation with additional features provided through the environment's info dictionary, which exposes metadata such as time-of-day and meal events.

- **Time of day:** Minutes since the start of the day, normalized by **1440**.
- **Meal flag:** A binary indicator—**1** if a meal occurs at the current step, **0** otherwise.
- **CGM slope:** The change in normalized CGM from the previous timestep.

**Actions:** The agent selects a **continuous insulin infusion rate** at each timestep. The default SimGlucose action space ranges from **0 to 30 U/min**, but in our setup we **limit the action range to 0–1 U/min**, following the design choice used in Paper 1. Paper 2 similarly restricts the dosing range to **0–5 U/min** to improve stability and safety.

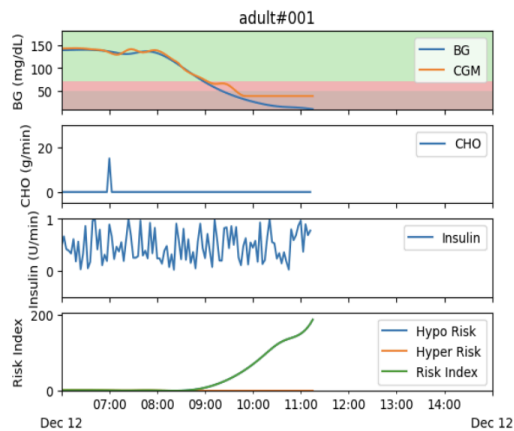
**Rewards:** The default SimGlucose reward is a binned stepwise signal: positive when glucose is in range, and negative when it is above or below range. We extend this by adding a smooth distance-to-optimal blood glucose (110 mg/dL) penalty, along with a small bonus when glucose moves toward the optimal glucose level. We also include mild asymmetric penalties, since being under range is clinically more dangerous and will terminate the episode. Strong penalties can cause the agent to learn no-action

behaviors, so we keep them small. Clinical risk-index rewards, commonly used in prior work, offer a more principled alternative and could be incorporated in future versions of this project.

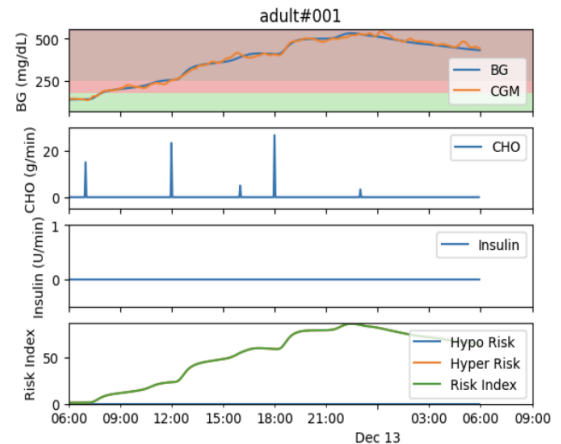
#### 4. Description of Technical Approach

Before introducing our PPO agents, we examine simple baseline dosing strategies. In one baseline, the agent takes random insulin actions within  $[0,1]$  at each timestep; as shown in the left figure, repeatedly dosing insulin drives glucose too low, causing hypoglycemia ( $BG < 70$  mg/dL) and early termination (death) when glucose falls below **10 mg/dL**, which is clinically unsafe. In contrast, the zero-insulin baseline shown in the right figure causes glucose to rise steadily toward the simulator's maximum, but the episode does not terminate since hyperglycemia ( $> 180$  mg/dL) is not treated as a failure condition. Thus, there is a need for a learned policy that can dose insulin appropriately, allowing the patient to survive the full 24-hour episode while remaining in the healthy glucose range (70-180 mg/LI) as much as possible.

--- Summary ---  
Steps survived: 106 (318 min, 5.30 hours)  
Time In Range (70-180 mg/dL): 55.66%  
Time Below Range ( $<70$  mg/dL): 44.34%  
Time Above Range ( $>180$  mg/dL): 0.00%



--- Summary ---  
Steps survived: 480 (1440 min, 24.00 hours)  
Time In Range (70-180 mg/dL): 8.12%  
Time Below Range ( $<70$  mg/dL): 0.00%  
Time Above Range ( $>180$  mg/dL): 91.88%



#### Baseline Proximal Policy Optimization (PPO):

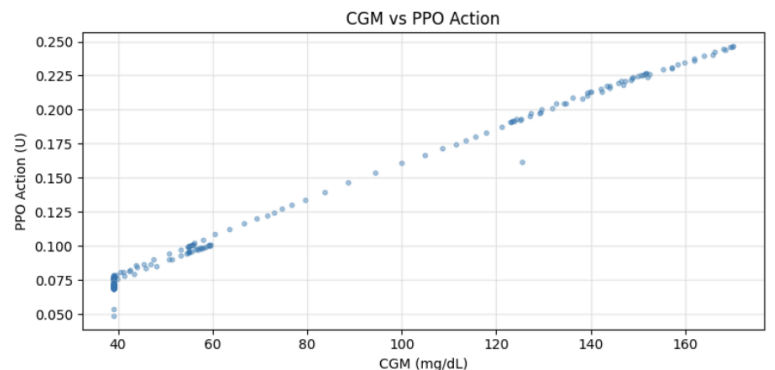
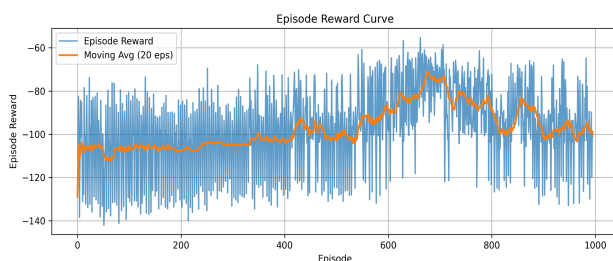
In the baseline setting, the agent receives a **4-dimensional observation** at each timestep which captures the current physiological state but provides no explicit memory of past observations.

obs = [normalized CGM, normalized time-of-day, meal flag, normalized CGM slope]

Because baseline PPO operates on a single observation vector, it often learns a largely reactive policy that maps the current CGM value directly to an insulin dose. In many runs, the policy continuously infuses a relatively steady insulin rate within a limited range (0.07-0.25 U/min), with higher CGM values leading to increased dosing and lower values leading to reduced dosing. This produces a roughly monotonic input-output relationship, as shown in the figure below, resembling a learned proportional controller rather than a policy that explicitly reasons over longer temporal dynamics.

The PPO agent was trained for 100,000 total timesteps with four parallel environments (25,000 steps per environment).

Figure below shows the episode return over training, where each point corresponds to the cumulative reward of a completed episode across the parallel environments.

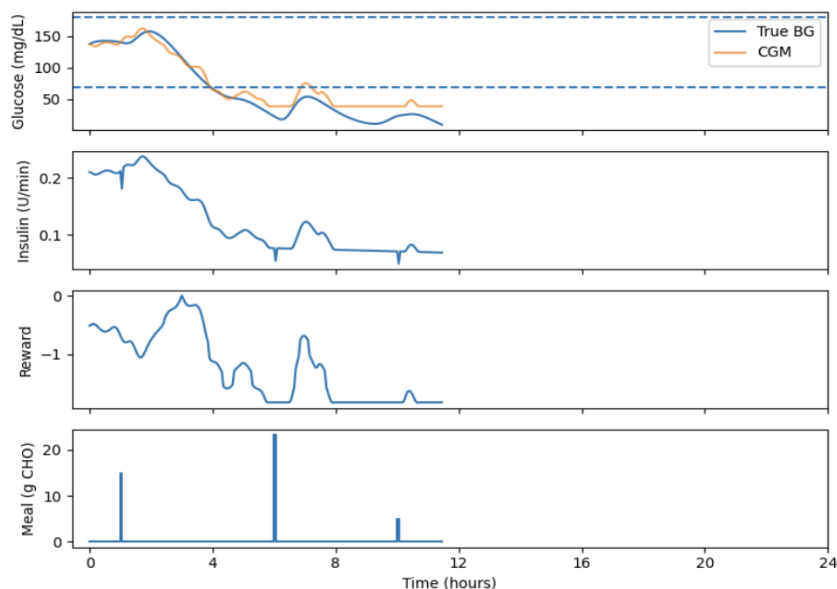


**Evaluation:** Evaluation is performed on the training patient for a single 24-hour simulated episode with fixed meals and some stochasticity from sensor noise and the insulin pump. In this policy, insulin is continuously infused, rapidly driving blood glucose downward and causing early termination when true blood glucose reaches 10 mg/dL. The patient survives for approximately 11.5 hours, during which 64.3% of the time is spent below the healthy range, with no time spent above range.

The policy also reacts to the boolean meal announcement feature. In this run, it responds by slightly decreasing insulin immediately when a meal is announced. I think the reason might be that when glucose is low, a meal will naturally raise blood glucose, and increasing insulin at the same time would counteract this effect. As a result, the policy may briefly lower insulin to allow glucose to recover safely.

==== Simulation Stats =====

Mean BG : 66.817  
 Min BG : 9.964  
 Max BG : 156.325  
 Time in Range (%) : 35.652  
 Time Below Range (%) : 64.348  
 Time Above Range (%) : 0.000  
 Total Insulin : 28.207  
 Total Reward : -269.296



### History Augmented PPO:

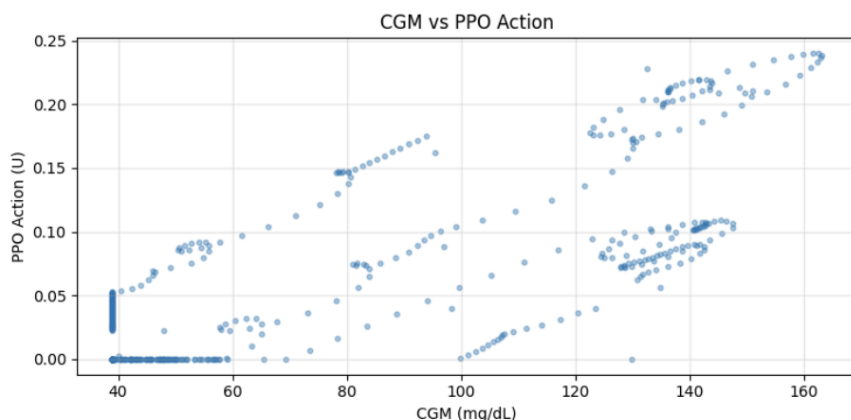
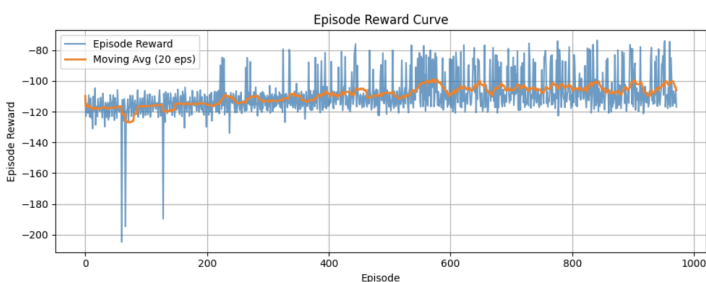
To incorporate short-term temporal context, we use a stacked observation wrapper. With a stack size of  $k=4$ , the observation has shape (16,) and consists of the last four observations concatenated together, where each single observation had shape (4,).

Stacked obs = [ obs\_{t-3}, obs\_{t-2}, obs\_{t-1}, obs\_t ]

the wrapper maintains a fixed-length buffer implemented using a `deque` with maximum size  $k$ . At each timestep, the current observation is appended to the buffer and the oldest observation is automatically discarded. The buffered observations are then concatenated to form the stacked observation. This provides the agent with short-term temporal context, enabling it to infer trends and delayed effects without using a recurrent model.

The stacked PPO policy leverages short-term glucose trends when selecting actions. As a result, the same CGM value can lead to different insulin doses depending on recent observation history, and the policy is not restricted to a simple linear mapping.

The stacked PPO agent was trained for 100,000 total timesteps with four parallel environments



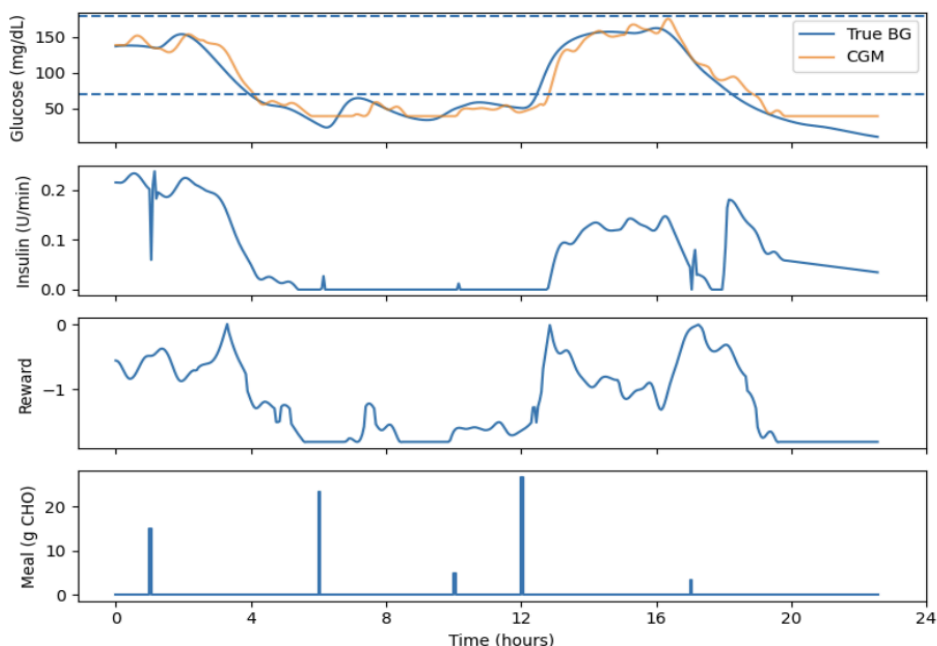
**Evaluation:** Evaluation is performed on the training patient using the stacked PPO policy for a single 24-hour simulated episode with fixed meals and some stochasticity from sensor noise and the insulin pump. In this run, the policy often gives zero insulin when glucose is in the low range, reducing the risk of glucose dropping to the termination threshold. As a result, the patient survives for approximately 23 hours. However, only 43% of the time is spent in the desired range, with 57% below range.

The total reward is worse than that of the baseline PPO, likely because the longer survival time allows the agent to accumulate negative rewards over a longer episode.

The policy also responds to boolean meal announcements, sometimes showing a brief decrease in insulin followed by an increase, and in other cases only a very minimal increase in insulin in meal times.

==== Simulation Stats =====

|                      |            |
|----------------------|------------|
| Mean BG              | : 81.052   |
| Min BG               | : 9.956    |
| Max BG               | : 162.468  |
| Time In Range (%)    | : 42.920   |
| Time Below Range (%) | : 57.080   |
| Time Above Range (%) | : 0.000    |
| Total Insulin        | : 31.879   |
| Total Reward         | : -543.491 |



Due to computational constraints and training instability, experiments were limited to 100k timesteps; while many prior studies train for 500k–1M timesteps, longer runs in this setting often led to policy collapse or near-constant action behaviors. To mitigate premature collapse and encourage exploration, an entropy coefficient in the range 0.1–0.3 was used for both PPO-based models.

#### Recurrent PPO:

At the start of the project, I chose to keep the approach simple and focus on standard PPO and stacked observations. However, around the time of the in-class presentation, the learned policies often converged to nearly constant insulin dosing with very small variation across different states, which raised concerns about achieving meaningful results. As a result, I started exploring recurrent PPO using the Stable-Baselines3 Contrib implementation, motivated by paper 1 and a [Reddit post](#) which mentioned Mlp Lstm policy helped. In practice, training was significantly slower, and the policy frequently collapsed to near-zero actions, leading to poor performance. Consequently, recurrent PPO is not included in the final evaluation of this report, as training is not yet stable; however, it remains an active work in progress and will be further explored in the very near term. Adding features substantially helped to solve the issue.

## 4. Software

a ) I wrote the code for wrappers, and scripts for data analysis and eval. All models are from stable-baseline.

b) code for setting up env, meals, connecting to gymnasium was borrowed from simglucose readme / repo

Readme : <https://github.com/jxx123/simglucose/blob/master/README.md> includes setting custom meal schedule

Setting up gymnasium : [https://github.com/jxx123/simglucose/blob/master/examples/run\\_gymnasium.py](https://github.com/jxx123/simglucose/blob/master/examples/run_gymnasium.py) includes setup, and the code for visual simulation is from internal simglucose code.

Here is an overall structure :

```
DiabetesRL/
├── src/
│   ├── wrappers.py          # include wrappers for clipping action range, future engineering and normalization, and stacking observations
│   ├── make_environment.py  # include function for building the regular env and the env with stacked observations
│   ├── reward_function.py   # includes the reward function
│   └── utils.py             # includes some utils for display, eval, experiments, etc
├── projcl.ipynb             # main notebook for evaluations/experiments
├── trail.ipynb              # notebook for training models
├── requirements.txt
└── readme.md
```

## 5. Experiments and Evaluation [at least 1 page]

We compare baseline PPO and stacked PPO using survival time as the primary metric, alongside random-insulin and constant-insulin baselines (e.g., 0.2 and 0.1 U/min). For each policy, we run 20 simulated episodes (20 days) on the same patient used for training(Adult #001), with each episode initialized using a different random seed to control for variation across runs. Performance is measured by the number of steps survived before termination, along with the corresponding minutes/ hours, and the standard deviation of survival time across episodes.

Random and constant-insulin policies show limited survival, with higher constant insulin rates failing earlier and lower rates surviving longer but remaining inflexible. In contrast, stacked PPO achieves the longest survival with higher variability, indicating adaptive behavior improved survival across episodes.

```
=== Survival results: PPO ===
Episodes: 20
Mean steps: 231.0 (693.0 min, 11.55 hours)
Std steps: 0.7
Min steps: 229
Max steps: 232
```

```
=== Survival results: STACKED PPO ===
Episodes: 20
Mean steps: 464.9 (1394.6 min, 23.24 hours)
Std steps: 12.3
Min steps: 441
Max steps: 480
```

```
=== Survival results: RANDOM ===
Episodes: 20
Mean steps: 103.2 (309.5 min, 5.16 hours)
Std steps: 3.9
Min steps: 88
Max steps: 107
```

```
=== Survival results: CONSTANT (CONST=0.2) ===
Episodes: 20
Mean steps: 174.0 (522.0 min, 8.70 hours)
Std steps: 0.0
Min steps: 174
Max steps: 174
```

```
=== Survival results: CONSTANT (CONST=0.1) ===
Episodes: 20
Mean steps: 242.0 (726.0 min, 12.10 hours)
Std steps: 0.0
Min steps: 242
Max steps: 242
```

```
=== Survival results: CONSTANT (CONST=0.08) ===
Episodes: 20
Mean steps: 366.0 (1098.0 min, 18.30 hours)
Std steps: 0.0
Min steps: 366
Max steps: 366
```

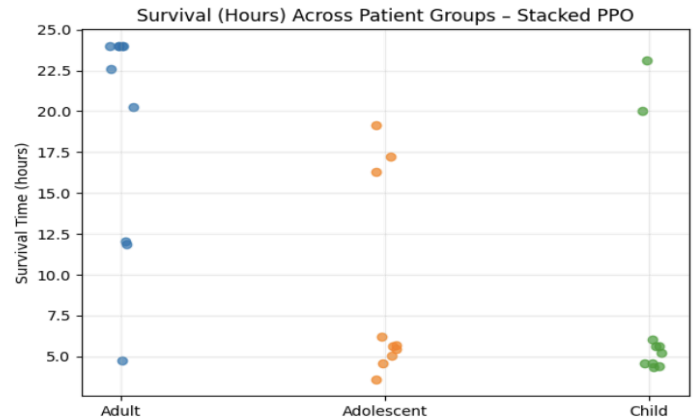
**Cross patient generalizability test :** SimGlucose provides 30 virtual patients (10 adults, 10 adolescents, and 10 children). Due to limited time and compute, I trained the policy on a single adult patient. This experiment evaluates the **generalization** of the adult-trained policy to **out-of-sample patients**. Specifically, the trained stacked PPO policy is evaluated on the training adult, the 9 other unseen adult patients, and all 10 unseen adolescent and 10 unseen child patients. For each patient, evaluation is performed using a single 24-hour simulated episode.

The plot shows survival time (in hours) across patients.

The policy performs well on most other adult patients but performs poorly on the majority of adolescent and child patients. This suggests that the policy is likely overfit to the training adult.

These results highlight the need to train on a more diverse set of patients, include patients from different age groups, to improve generalizability or to develop personalized models tailored to specific patient subgroups.

Below is the mean survival time and glucose statistics averaged across each patient group; adult patients survive on average about 19 hours, while both adolescent and child patients survive only around 8 hours.



**Adult** | steps 383.00 | hours 19.15 | TIR 48.9 | TBR 47.5 | TAR 3.7

**Adolescent** | steps 177.60 | hours 8.88 | TIR 55.0 | TBR 40.2 | TAR 4.8

**Child** | steps 166.90 | hours 8.35 | TIR 39.8 | TBR 57.6 | TAR 2.6

## 6. Discussion and Conclusion

After the in class progress report, adding extra features such as CGM slope, meal announcements, and time-of-day helped reduce partial observability and enabled the agent to react to meals; however, the learned policies are still not perfect, as the stacked ppo policy survives the episode but does not achieve optimal time-in-range.

Training was highly unstable due to the stochastic environment and inherent partial observability, and outcomes varied across runs despite setting seeds where possible. As a result, the results shown correspond to a representative policy, and improving robustness and consistency remains a short-term direction for future work.

This project highlighted the difficulty of applying reinforcement learning to automated insulin dosing under partial observability and delayed dynamics. One surprising result was how differently the learned policies behaved compared to my initial intuition. I expected the agent to give no insulin when glucose was below range and to increase dosing mainly when glucose was high; however, only the stacked PPO occasionally learned to give zero insulin, and only in some runs. In contrast, the vanilla PPO often learned to apply insulin at a relatively steady rate across states. It remains unclear to me whether insulin should be continuously given at every time step or not. Prior work reports differing strategies: one paper explicitly defined a safe-control region to suspend insulin delivery below a safety threshold (Marchetti, 2025) [4], while another paper demonstrates 24-hour survival by almost continuously injecting low insulin rates between 0–0.1 U/min over 24 hours (Zhao, 2025) [5].

A key limitation of this approach is limited generalization, models trained on a single adult patient failed to perform well on adolescent and child patients, highlighting how strongly physiological differences affect policy behavior and underscoring the need for patient-specific training or broader population-level learning. The approach also relies on meal announcements, which require manual user input and prevent fully automated control, as noted in paper2. Also, the reward function is relatively simple, which may limit optimal glucose regulation compared to clinically defined reward functions.

**Future Work:** After finalizing recurrent PPO, future work could explore clinically defined risk-based reward functions and training on multiple patients to improve robustness and generalization. Alternative models such as SAC are also noted in multiple papers.

In the very long term, I would like to explore model-based reinforcement learning approaches that learn latent representations of the glucose–insulin environment, capturing underlying physiological dynamics to further improve performance and enable more personalized control.

### References

- [1] Xie, Jinyu “JXX123/Simglucose: A Type-1 Diabetes Simulator Implemented in Python for Reinforcement Learning Purpose.” *GitHub*, [github.com/jxx123/simglucose](https://github.com/jxx123/simglucose).
- [2] Youssefpour, Parsa. *Explainable Recurrent Proximal Policy Optimization: Determining Proper Insulin Dosing*, [github.com/Pyoussefpour/Interpretable\\_RL/blob/main/Explainable\\_RPPO.pdf](https://github.com/Pyoussefpour/Interpretable_RL/blob/main/Explainable_RPPO.pdf).
- [3] Hettiarachchi, Chirath, et al. “G2P2C — a modular reinforcement learning algorithm for glucose control by glucose prediction and planning in type 1 diabetes.” *Biomedical Signal Processing and Control*, vol. 90, Apr. 2024, p. 105839, <https://doi.org/10.1016/j.bspc.2023.105839>.
- [4] Marchetti, Alessandro, et al. “Deep Reinforcement Learning for Type 1 diabetes: Dual PPO controller for personalized insulin management.” *Computers in Biology and Medicine*, vol. 191, June 2025, p. 110147, <https://doi.org/10.1016/j.combiomed.2025.110147>.
- [5] Zhao, Yan Feng, et al. “A safe-enhanced fully closed-loop artificial pancreas controller based on Deep Reinforcement Learning.” *PLOS ONE*, vol. 20, no. 1, 27 Jan. 2025, <https://doi.org/10.1371/journal.pone.0317662>.